

Social Media Insight using Naive Bayes

Text-based documents contain lots of information. Examples include books, legal documents, social media, and e-mail. Extracting information from text-based documents is critically important to modern AI systems, for example in search engines, legal AI, and automated news services.

Extraction of useful features from text is a difficult problem. Text is not numerical in nature, therefore a model must be used to create features that can be used with data mining algorithms. The good news is that there are some simple models that do a great job at this, including the bag-of-words model that we will use in this chapter.

In this chapter, we look at extracting features from text for use in data mining applications. The specific problem we tackle in this chapter is term disambiguation on social media - determining which meaning a word has based on its context.

We will cover the following topics in this chapter:

- Downloading data from social network APIs
- Transformers and models for text data
- The Naive Bayes classifier
- Using JSON for saving and loading datasets
- The NLTK library for feature creation
- The F-measure for evaluation

Disambiguation

Text data is often called an *unstructured format*. There is a lot of information in text, but it is just *there*; no headings, no required format (save for normal grammatical rules), loose syntax, and other problems prohibit the easy extraction of information from text. The data is also highly connected, with lots of mentions and cross-references—just not in a format that allows us to easily extract it! Even seemingly easy problems, such as determining if a word is a noun, have lots of weird edge cases that make it difficult to do reliably.

We can compare the information stored in a book with that stored in a large database to see the difference. In the book, there are characters, themes, places, and lots of information. However, a book needs to be read and interpreted, with cultural context, to gain this information. In contrast, a database sits on your server with column names and data types. All the information is there and the level of interpretation needed to extract specific information is quite low.

Information about the data, such as its type or its meaning, is called metadata. Text lacks metadata. A book also contains some metadata in the form of a table of contents and index but the degree of information included in these sections is significantly lower than that of a database.

One of the problems in working with text is **term disambiguation**. When a person uses the word *bank*, is this a financial message or an environmental message (such as river bank)? This type of disambiguation is quite easy in many circumstances for humans (although there are still troubles), but much harder for computers to do.

In this chapter, we will look at disambiguating the use of the term Python on Twitter's stream. When people talk about Python, they could be talking about the following things:

- The programming language Python
- Monty Python, the classic comedy group
- The snake Python
- A make of shoe called Python

There can be many other things called Python. The aim of our experiment is to take a tweet mentioning Python and determine whether it is talking about the programming language, based only on the content of the tweet.



A message on Twitter is called a tweet and is limited to 140 characters. Tweets include lots of metadata, such as the time and date of posting, who posted it, and so on. However in regards to the topic of the tweet, there is not much in this regard.

In this chapter, we are going to perform a data mining experiment consisting of the following steps:

1. Download a set of tweets from Twitter.
2. Manually classify them to create a dataset.
3. Save the dataset so that we can replicate our research.
4. Use the Naive Bayes classifier to create a classifier to perform term disambiguation.

Downloading data from a social network

We are first going to download a corpus of data from Twitter and use it to sort out spam from useful content. Twitter provides a robust API for collecting information from its servers and this API is free for small-scale usage. It is, however, subject to some conditions that you'll need to be aware of if you start using Twitter's data in a commercial setting.

First, you'll need to sign up for a Twitter account (which is free). Go to <http://twitter.com> and register an account if you do not already have one.

Next, you'll need to ensure that you only make a certain number of requests per minute. This limit is currently 15 requests per 15 minutes (it depends on the exact API). It can be tricky ensuring that you don't breach this limit, so it is highly recommended that you use a library to talk to Twitter's API.



If you are using your own code (that is making the web calls with your own code) to connect with a web-based API, ensure that you read the documentation about rate limiting their documentation and understand the limitations. In Python, you can use the time library to perform a pause between calls to ensure you do not breach the limit.

You will then need a key to access Twitter's data. Go to <http://twitter.com> and sign in to your account. When you are logged in, go to <https://apps.twitter.com/> and click on Create New App. Create a name and description for your app, along with a website address. If you don't have a website to use, insert a placeholder. Leave the Callback URL field blank for this app—we won't need it. Agree to the terms of use (if you do) and click on Create your Twitter application.

Keep the resulting website open—you'll need the access keys that are on this page. Next, we need a library to talk to Twitter. There are many options; the one I like is simply called `twitter`, and is the official Twitter Python library.



You can install twitter using pip3 install twitter (on the command line) if you are using pip to install your packages. At the time of writing, Anaconda does not include twitter, therefore you can't use conda to install it. If you are using another system or want to build from source, check the documentation at <https://github.com/sixohsix/twitter>

Create a new Jupyter Notebook to download the data. We will create several notebooks in this chapter for various different purposes, so it might be a good idea to also create a folder to keep track of them. This first notebook, `ch6_get_twitter`, is specifically for downloading new Twitter data.

First, we import the `twitter` library and set our authorization tokens. The consumer key and consumer secret will be available on the Keys and Access Tokens tab on your Twitter app's page. To get the access tokens, you'll need to click on the Create my access token button, which is on the same page. Enter the keys into the appropriate places in the following code:

```
import twitter
consumer_key = "<Your Consumer Key Here>"
consumer_secret = "<Your Consumer Secret Here>"
access_token = "<Your Access Token Here>"
access_token_secret = "<Your Access Token Secret Here>"
authorization = twitter.OAuth(access_token, access_token_secret, consumer_key, consumer_secret)
```

We are going to get our tweets from Twitter's search function. We will create a reader that connects to twitter using our authorization, and then use that reader to perform searches. In the Notebook, we set the filename where the tweets will be stored:

```
import os
output_filename = os.path.join(os.path.expanduser("~"), "Data", "twitter", "python_tweets.json")
```

Next, create an object that can read from Twitter. We create this object with our authorization object that we set up earlier:

```
t = twitter.Twitter(auth=authorization)
```

We then open our output file for writing. We open it for appending—this allows us to rerun the script to obtain more tweets. We then use our Twitter connection to perform a search for the word Python. We only want the statuses that are returned for our dataset. This code takes the tweet, uses the json library to create a string representation using the dumps function, and then writes it to the file. It then creates a blank line under the tweet so that we can easily distinguish where one tweet starts and ends in our file:

```
import json
with open(output_filename, 'a') as output_file:
    search_results = t.search.tweets(q="python", count=100)['statuses']
    for tweet in search_results:
        if 'text' in tweet:
            output_file.write(json.dumps(tweet))
            output_file.write("\n\n")
```

In the preceding loop, we also perform a check to see whether there is text in the tweet or not. Not all of the objects returned by twitter will be actual tweets (for example, some responses will be actions to delete tweets). The key difference is the inclusion of text as a key, which we test for. Running this for a few minutes will result in 100 tweets being added to the output file.



You can keep re-running this script to add more tweets to your dataset, keeping in mind that you may get some duplicates in the output file if you rerun it too fast (that is before Twitter gets new tweets to return!). For our initial experiment, 100 tweets will be enough, but you will probably want to come back and rerun this code to get that up to about 1000.

Loading and classifying the dataset

After we have collected a set of tweets (our dataset), we need labels to perform classification. We are going to label the dataset by setting up a form in a Jupyter Notebook to allow us to enter the labels. We do this by loading the tweets we collected in the previous section, iterating over them and providing (manually) a classification on whether they refer to Python the programming language or not.

The dataset we have stored is nearly, but not quite, in a **JSON** format. JSON is a format for data that doesn't impose much structure on the contents, just on the syntax. The idea behind JSON is that the data is in a format directly readable in JavaScript (hence the name, *JavaScript Object Notation*). JSON defines basic objects such as numbers, strings, lists, and dictionaries, making it a good format for storing datasets, if they contain data that isn't numerical. If your dataset is fully numerical, you would save space and time using a matrix-based format like in NumPy.



A key difference between our dataset and real JSON is that we included newlines between tweets. The reason for this was to allow us to easily append new tweets (the actual JSON format doesn't allow this easily). Our format is a JSON representation of a Tweet, followed by a newline, followed by the next Tweet, and so on.

To parse it, we can use the json library but we will have to first split the file by newlines to get the actual tweet objects themselves. Set up a new Jupyter Notebook, I called mine `ch6_label_twitter`. Within it, we will first load the data from our input filename by iterating over the file, storing tweets as we loop. The code below does a basic check that there is actual text in the tweet. If it does, we use the json library to load the tweet and then we add it to a list:

```
import json
import os

# Input filename
input_filename = os.path.join(os.path.expanduser("~"), "Data", "twitter", "python_tweets.json")

# Output filename
labels_filename = os.path.join(os.path.expanduser("~"), "Data", "twitter", "python_classes.json")

tweets = []
with open(input_filename) as inf:
    for line in inf:
        if len(line.strip()) == 0:
            continue
        tweets.append(json.loads(line))
```

We are now interested in manually classifying whether an item is relevant to us or not (in this case, relevant means refers to the programming language Python). We will use the Jupyter Notebook's ability to embed HTML and talk between JavaScript and Python to create a viewer of tweets to allow us to easily and quickly classify the tweets as spam or not. The code will present a new tweet to the user (you) and ask for a label: *is it relevant or not?* It will then store the input and present the next tweet to be labeled.

First, we create a list for storing the labels. These labels will be stored whether or not the given tweet refers to the programming language Python, and it will allow our classifier to learn how to differentiate between meanings.

```

<div name="tweetbox">
  Instructions: Click in text box. Enter a 1 if the tweet is relevant, enter 0 otherwise.<br>
  Tweet: <div id="tweet_text" value="text"></div><br>
  <input type="text" id="capture"></input><br>
</div>

<script>
function set_label(label){
  var kernel = IPython.notebook.kernel;
  kernel.execute("labels.append(" + label + ")");
  load_next_tweet();
}

function load_next_tweet(){
  var code_input = "get_next_tweet()";
  var kernel = IPython.notebook.kernel;
  var callbacks = { 'iopub' : {'output' : handle_output}};
  kernel.execute(code_input, callbacks, {silent:false});
}

function handle_output(out){
  console.log(out);
  var res = out.content.data["text/plain"];
  $("#div#tweet_text").html(res);
}

$("#input#capture").keypress(function(e) {
  console.log(e);
  if(e.which == 48) {
    // 0 pressed
    set_label(0);
    $("#input#capture").val("");
  }else if (e.which == 49){
    // 1 pressed
    set_label(1);
    $("#input#capture").val("");
  }
});

load_next_tweet();
</script>

```

You will need to enter all of this code into a single cell (or copy it from the code bundle). It contains the mix of HTML and JavaScript necessary to get input from you to manually classify the tweets. If you need to stop or save your progress, run the following code in the next cell. It will save your progress (and doesn't interrupt the above HTML code either, which can be left running):

```

with open(labels_filename, 'w') as outf:
    json.dump(labels, outf)

```

We also check if we have any labels already and load them. This helps if you need to close the notebook down midway through labeling. This code will load the labels from where you left off. It is generally a good idea to consider how to save at midpoints for tasks like this. Nothing hurts quite like losing an hour of work because your computer crashed before you saved the labels! The code to do this loading follows:

```
labels = []
if os.path.exists(labels_filename):
    with open(labels_filename) as inf:
        labels = json.load(inf)
```

The first time you run this, nothing will happen. After manually classifying some examples you can save your progress and then close the Notebook. After that, you can reopen the Notebook and return to where you were up to.



If you make one or two mistakes classifying, don't worry too much. If you make lots of mistakes and want to start again, delete just `python_classes.json` and the above code will pick up with an empty set of classifications. If you need to delete all of your data and start again with new tweets, make sure to delete (or move) both files - `python_tweets.json` and `python_classes.json`. Otherwise, this Notebook will get confused, giving classes from the old dataset to the new tweets.

Next, we create a simple function that will return the next tweet that needs to be labeled. We can work out which is the next tweet by finding the first one that hasn't yet been labeled. The code is pretty straight-forward. We determine how many tweets we have labeled (with `len(labels)`), and get the next tweet in the `tweet_sample` list:

```
def get_next_tweet():
    return tweets[len(labels)][ 'text' ]
```

The next step in our experiment is to collect information from the user (you!) on which tweets are referring to Python (the programming language) and which are not.



As of yet, there is not a good, straightforward way to get interactive feedback with pure Python in Jupyter Notebooks for such a large number of text documents. For this reason, we will use some JavaScript and HTML to get this input from the user. There are many ways to do this, below is just one example.

To get the feedback, we need a JavaScript component to load the next tweet and show it. We also need a HTML component to create the HTML elements to display that tweet. I won't go into the details of the code here, except to give this general workflow:

1. Obtain the next tweet that needs to be classified with `load_next_tweet`
2. Show it to the user with `handle_output`
3. Wait for the user to press either 0 or 1 with `$("#input#capture").keypress`
4. Store that result in the classes list with `set_label`

This keeps happening until we reach the end of the list (at which point an `IndexError` occurs, indicating we have no more tweets to classify). The code is below (remember that you can get the code from Packt or from the official GitHub repository):

```
%%html
```


Creating a replicable dataset from Twitter

In data mining, there are lots of variables. These aren't the parameters of the data mining algorithms - they are the methods of data collection, how the environment is set up, and many other factors. Being able to replicate your results is important as it enables you to verify or improve upon your results.

Getting 80 percent accuracy on one dataset with algorithm X, and 90 percent accuracy on another dataset with algorithm Y doesn't mean that Y is better. We need to be able to test on the same dataset in the same conditions to be able to properly compare. With running the preceding code, you will get a different dataset to the one I created and used. The main reasons are that Twitter will return different search results for you than me based on the time you performed the search.

Even after that, your labeling of tweets might be different from what I do. While there are obvious examples where a given tweet relates to the python programming language, there will always be gray areas where the labeling isn't obvious. One tough gray area I ran into was tweets in non-English languages that I couldn't read. In this specific instance, there are options in Twitter's API for setting the language, but even these aren't going to be perfect.

Due to these factors, it is difficult to replicate experiments on databases that are extracted from social media, and Twitter is no exception. Twitter explicitly disallows sharing datasets directly. One solution to this is to share tweet IDs only, which you can share freely. In this section, we will first create a tweet ID dataset that we can freely share. Then, we will see how to download the original tweets from this file to recreate the original dataset. First, we save the replicable dataset of tweet IDs.

After creating another new Jupyter Notebook, first set up the filenames as before. This is done in the same way we did labeling but there is a new filename where we can store the replicable dataset. The code is as follows:

```
import os
input_filename = os.path.join(os.path.expanduser("~"), "Data", "twitter", "python_tweets.json")
labels_filename = os.path.join(os.path.expanduser("~"), "Data", "twitter", "python_classes.json")
replicable_dataset = os.path.join(os.path.expanduser("~"), "Data", "twitter", "replicable_dataset.json")
```

We load the tweets and labels as we did in the previous notebook:

```
import json
tweets = []
with open(input_filename) as inf:
    for line in inf:
        if len(line.strip()) == 0:
            continue
        tweets.append(json.loads(line))
if os.path.exists(labels_filename):
```

```
actual_labels = []
label_mapping = dict(tweet_ids)
```

Next, we are going to create a twitter server to collect all of these tweets. This is going to take a little longer. Import the twitter library that we used before, creating an authorization token and using that to create the twitter object:

```
import twitter
consumer_key = "<Your Consumer Key Here>"
consumer_secret = "<Your Consumer Secret Here>"
access_token = "<Your Access Token Here>"
access_token_secret = "<Your Access Token Secret Here>"
authorization = twitter.OAuth(access_token, access_token_secret, consumer_key, consumer_secret)
t = twitter.Twitter(auth=authorization)
```

Next, we will loop through each of the tweet ids, and ask twitter to recover the original tweet. A good feature of twitter's API is that we can ask for 100 tweets at a time, drastically reducing the number of API calls. Interestingly, from twitter's point of view, it is the same number of calls to get one tweet or 100 tweets, as long as its a single request.

The following code will loop through our tweets in groups of 100, join together the id values, and get the tweet information for each of them.

```
all_ids = [tweet_id for tweet_id, label in tweet_ids]
with open(tweet_filename, 'a') as output_file:
    # We can lookup 100 tweets at a time, which saves time in asking twitter for them
    for start_index in range(0, len(all_ids), 100):
        id_string = ",".join(str(i) for i in all_ids[start_index:start_index+100])
        search_results = t.statuses.lookup(_id=id_string)
        for tweet in search_results:
            if 'text' in tweet:
                # Valid tweet - save to file
                output_file.write(json.dumps(tweet))
                output_file.write("\n\n")
                actual_labels.append(label_mapping[tweet['id']])
```

In this code, we then check each tweet to see if it is a valid tweet and then save it to our file if it is. Our final step is to save our resulting labels:

```
with open(labels_filename, 'w') as outf:
    json.dump(actual_labels, outf)
```

```
with open(labels_filename) as inf:
    labels = json.load(inf)
```

Now we create a dataset by looping over both the tweets and labels at the same time and saving those in a list. An important side-effect of this code is, by putting labels first in the zip function, it will only load enough tweets for the labels we have created. In other words, you can run this code on partially classified data:

```
dataset = [(tweet['id'], label) for label, tweet in zip(labels, tweets)]
```

Finally, we save the results in our file:

```
with open(replicable_dataset, 'w') as outf:
    json.dump(dataset, outf)
```

Now that we have the Tweet IDs and labels saved, we can recreate the original dataset. If you are looking to recreate the dataset I used for this chapter, it can be found in the code bundle that comes with this book. Loading the preceding dataset is not difficult but it can take some time.

Start a new Jupyter Notebook and set the dataset, label, and tweet ID filenames as before. I've adjusted the filenames here to ensure that you don't overwrite your previously collected dataset, but feel free to change these if you do want to override.

The code is as follows:

```
import os
tweet_filename = os.path.join(os.path.expanduser("~"), "Data", "twitter", "replicable_python_tw
labels_filename = os.path.join(os.path.expanduser("~"), "Data", "twitter", "replicable_python_c
replicable_dataset = os.path.join(os.path.expanduser("~"), "Data", "twitter", "replicable_datast
```

Next, load the tweet IDs from the file using JSON:

```
import json
with open(replicable_dataset) as inf:
    tweet_ids = json.load(inf)
```

Saving the labels is very easy. We just iterate through this dataset and extract the IDs. We could do this quite easily with just two lines of code (open file and save tweets). However, we can't guarantee that we will get all the tweets we are after (for example, some may have been changed to private since collecting the dataset) and therefore the labels will be incorrectly indexed against the data. As an example, I tried to recreate the dataset just one day after collecting them and already two of the tweets were missing (they might be deleted or made private by the user). For this reason, it is important to only print out the labels that we need.

To do this, we first create an empty actual labels list to store the labels for tweets that we actually recover from twitter, and then create a dictionary mapping the tweet IDs to the labels. The code is as follows:

Text transformers

Now that we have our dataset, how are we going to perform data mining on it?

Text-based datasets include books, essays, websites, manuscripts, programming code, and other forms of written expression. All of the algorithms we have seen so far deal with numerical or categorical features, so how do we convert our text into a format that the algorithm can deal with? There are a number of measurements that could be taken.

For instance, average word and average sentence length are used to predict the readability of a document. However, there are lots of feature types such as word occurrence which we will now investigate.

Bag-of-words models

One of the simplest but highly effective models is to simply count each word in the dataset. We create a matrix, where each row represents a document in our dataset and each column represents a word. The value of the cell is the frequency of that word in the document. This is known as the **bag-of-words model**.

Here's an excerpt from *The Lord of the Rings*, J.R.R. Tolkien:

*Three Rings for the Elven-kings under the sky,
Seven for the Dwarf-lords in halls of stone, Nine for Mortal Men, doomed to die,
One for the Dark Lord on his dark throne In the Land of Mordor where the Shadows lie.
One Ring to rule them all, One Ring to find them,
One Ring to bring them all and in the darkness bind them.
In the Land of Mordor where the Shadows lie.*

- J.R.R. Tolkien's epigraph to *The Lord of The Rings*

The word *the* appears nine times in this quote, while the words *in*, *for*, *to*, and *one* each appear four times. The word *ring* appears three times, as does the word *of*.

We can create a dataset from this, choosing a subset of words and counting the frequency:

Word	the	one	ring	to
Frequency	9	4	3	4

To do this for all words in a single document, we can use the **Counter** class. When counting words, it is normal to convert all letters to lowercase, which we do when creating the string. The code is as follows:

```
s = ""Three Rings for the Elven-kings under the sky, Seven for the Dwarf-lords in halls of st  
words = s.split()  
from collections import Counter  
c = Counter(words)  
print(c.most_common(5))
```

Printing `c.most_common(5)` gives the list of the top five most frequently occurring words. Ties are not handled well as only five are given and a very large number of words all share a tie for fifth place.

The bag-of-words model has three major types, with many variations and alterations.

- The first is to use the raw frequencies, as shown in the preceding example. This has the same drawback as any non-normalised data - words with high variance due to high overall values (such as) *the* overshadow lower frequency (and therefore lower-variance) words, even though the presence of the word *the* rarely has much importance.

- The second model is to use the normalized frequency, where each document's sum equals 1. This is a much better solution as the length of the document doesn't matter as much, but it still means words like the overshadow lower frequency words. The third type is to simply use binary features—a value is 1 if it occurs, and 0 otherwise. We will use binary representation in this chapter.
- Another (arguably more popular) method for performing normalization is called **term frequency-inverse document frequency (tf-idf)**. In this weighting scheme, term counts are first normalized to frequencies and then divided by the number of documents in which it appears in the corpus. We will use tf-idf in [Chapter 10](#), *Clustering News Articles*.

n-gram features

One variation on the standard bag-of-words model is called the n-gram model. An n-grams model addresses the deficiency of context in the bag-of-words model. With a bag-of-words model, only individual words are counted by themselves. This means that common word pairs, such as *United States*, lose meaning they have in the sentence because they are treated as individual words.

There are algorithms that can read a sentence, parse it into a tree-like structure, and use this to create very accurate representations of the meaning behind words. Unfortunately, these algorithms are computationally expensive. This makes it difficult to apply them to large datasets.

To compensate for these issues of context and complexity, the n-grams model fits into the middle ground. It has more context than the bag-of-words model, while only being slightly more expensive computationally.

An n-gram is a subsequence of n consecutive, overlapping, tokens. In this experiment, we use word n-grams, which are n-grams of word-tokens. They are counted the same way as a bag-of-words, with the n-grams forming a *word* that is put in the bag. The value of a cell in this dataset is the frequency that a particular n-gram appears in the given document.



The value of n is a parameter. For English, setting it to between 2 to 5 is a good start, although some applications call for higher values. Higher values for n result in sparse datasets, as when n increases it is less likely to have the same n -gram appear across multiple documents. Having $n=1$ results in simply the bag-of-words model.

As an example, for $n=3$, we extract the first few n-grams in the following quote:

Always look on the bright side of life.

The first n-gram (of size 3) is *Always look on*, the second is *look on the*, the third is *on the bright*. As you can see, the n-grams overlap and cover three words each. Word n-grams have advantages over using single words. This simple concept introduces some context to word use by considering its local environment, without a large overhead of understanding the language computationally.

A disadvantage of using n-grams is that the matrix becomes even sparser—word n-grams are unlikely to appear twice (especially in tweets and other short documents!). Specifically for social media and other short documents, word n-grams are unlikely to appear in too many different tweets, unless it is a retweet. However, in larger documents, word n-grams are quite effective for many applications. Another form of n-gram for text documents is that of a character n-gram. That said, you'll see shortly that word n-grams are quite effective in practice.

Rather than using sets of words, we simply use sets of characters (although character n-grams have lots of options for how they are computed!). This type of model can help identify words that are misspelled, as well as providing other benefits to classification. We will test character n-grams in this chapter and see them again in [Chapter 9, Authorship Attribution](#).

Other text features

There are other features that can be extracted too. These include syntactic features, such as the usage of particular words in sentences. Part-of-speech tags are also popular for data mining applications that need to understand meaning in text. Such feature types won't be covered in this book. If you are interested in learning more, I recommend *Python 3 Text Processing with NLTK 3 Cookbook*, Jacob Perkins, Packt publication.

There are a number of libraries for working with text data in Python. The most commonly known one is called the Natural Language ToolKit (NLTK). The scikit-learn library also has the CountVectorizer class that performs a similar action, and it is recommended you take a look at it (we will use it in [Chapter 9, Authorship Attribution](#)). NLTK has more features for word tokenization and part of speech tagging (that is identifying which words are nouns, verbs and so on).

The library we are going to use is called spaCy. It is designed from the ground up to be fast and reliable for natural language processing. It's less well-known than NLTK, but is rapidly growing in popularity. It also simplifies some of the decisions, but has a slightly more difficult syntax to use, compared to NLTK.



For production systems, I recommend using spaCy, which is faster than NLTK. NLTK was built for teaching, while spaCy was built for production. They have different syntaxes, meaning it can be difficult to port code from one library to another. If you aren't looking into experimenting with different types of natural language parsers, I recommend using spaCy.

Naive Bayes

Naive Bayes is a probabilistic model that is, unsurprisingly, built upon a naive interpretation of Bayesian statistics. Despite the naive aspect, the method performs very well in a large number of contexts. Because of the naive aspect, it works quite quickly. It can be used for classification of many different feature types and formats, but we will focus on one in this chapter: binary features in the bag-of-words model.

Understanding Bayes' theorem

For most of us, when we were taught statistics, we started from a **frequentist approach**. In this approach, we assume the data comes from some distribution and we aim to determine what the parameters are for that distribution. However, those parameters are (perhaps incorrectly) assumed to be fixed. We use our model to describe the data, even testing to ensure the data fits our model.

Bayesian statistics instead model how people (at least, non-frequentist statisticians) actually reason. We have some data, and we use that data to update our model about how likely something is to occur. In Bayesian statistics, we use the data to describe the model rather than using a model and confirming it with data (as per the frequentist approach).

It should be noted that frequentist statistics and Bayesian statistics ask and answer slightly different questions. A direct comparison is not always correct.

Bayes' theorem computes the value of $P(A|B)$. That is, knowing that B has occurred, what is the probability of event A occurring. In most cases, B is an observed event such as *it rained yesterday*, and A is a prediction it will rain today. For data mining, B is usually *we observed this sample* and A is *does the sample belong to this class* (the class prediction). We will see how to use Bayes' theorem for data mining in the next section.

The equation for Bayes' theorem is given as follows:

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}.$$

As an example, we want to determine the probability that an e-mail containing the word *drugs* is spam (as we believe that such a tweet may be a pharmaceutical spam).

A , in this context, is the probability that this tweet is spam. We can compute $P(A)$, called the prior belief directly from a training data by computing the percentage of tweets in our dataset that are spam. If our dataset contains 30 spam messages for every 100 e-mails, $P(A)$ is 30/100 or 0.3.

B , in this context, is this tweet contains the word *drugs*. Likewise, we can compute $P(B)$ by computing the percentage of tweets in our dataset containing the word *drugs*. If 10 e-mails in every 100 of our training dataset contain the word *drugs*, $P(B)$ is 10/100 or 0.1. Note that we don't care if the e-mail is spam or not when computing this value.

$P(B|A)$ is the probability that an e-mail contains the word *drugs* if it is spam. This is also easy to compute from our training dataset. We look through our training set for spam e-mails and compute the percentage of them that contain the word *drugs*. Of our 30 spam e-mails, if 6 contain the word *drugs*, then $P(B|A)$ is calculated as 6/30 or 0.2.

From here, we use Bayes' theorem to compute $P(A|B)$, which is the probability that a tweet containing the word *drugs* is

spam. Using the previous equation, we see the result is 0.6. This indicates that if an e-mail has the word drugs in it, there is a 60 percent chance that it is spam.



Note the empirical nature of the preceding example—we use evidence directly from our training dataset, not from some preconceived distribution. In contrast, a frequentist view of this would rely on us creating a distribution of the probability of words in tweets to compute similar equations.

Naive Bayes algorithm

Looking back at our Bayes' theorem equation, we can use it to compute the probability that a given sample belongs to a given class. This allows the equation to be used as a classification algorithm.

With C as a given class and D as a sample in our dataset, we create the elements necessary for Bayes' theorem, and subsequently Naive Bayes. Naive Bayes is a classification algorithm that utilizes Bayes' theorem to compute the probability that a new data sample belongs to a particular class.

$P(D)$ is the probability of a given data sample. It can be difficult to compute this, as the sample is a complex interaction between different features, but luckily it is constant across all classes. Therefore, we don't need to compute it at all, as all we do in the final step is compare relative values.

$P(D|C)$ is the probability of the data point belonging to the class. This could also be difficult to compute due to the different features. However, this is where we introduce the naive part of the Naive Bayes algorithm. We naively assume that each feature is independent of each other. Rather than computing the full probability of $P(D|C)$, we compute the probability of each feature $D1, D2, D3, \dots$ and so on. Then, we just multiply them together:

$$P(D|C) = P(D1|C) \times P(D2|C) \dots \times P(Dn|C)$$

Each of these values is relatively easy to compute with binary features; we simply compute the percentage of times it is equal in our sample dataset.



In contrast, if we were to perform a non-naive Bayes version of this part, we would need to compute the correlations between different features for each class. Such computation is infeasible at best, and nearly impossible without vast amounts of data or adequate language analysis models.

From here, the algorithm is straightforward. We compute $P(C|D)$ for each possible class, ignoring the $P(D)$ term entirely. Then we choose the class with the highest probability. As the $P(D)$ term is consistent across each of the classes, ignoring it has no impact on the final prediction.

How it works

As an example, suppose we have the following (binary) feature values from a sample in our dataset: [0, 0, 0, 1].

Our training dataset contains two classes with 75 percent of samples belonging to the class 0, and 25 percent belonging to the class 1. The likelihood of the feature values for each class are as follows:

For class 0: [0.3, 0.4, 0.4, 0.7]

For class 1: [0.7, 0.3, 0.4, 0.9]

These values are to be interpreted as: for feature 1, it has a value of 1 in 30 percent of cases for samples with class 0. It is a value of 1 in 70 percent of samples with class 1.

We can now compute the probability that this sample should belong to the class 0. $P(C=0) = 0.75$ which is the probability that the class is 0. Again, $P(D)$ isn't needed for the Naive Bayes algorithm and is simply removed from the equation. Let's take a look at the calculation:

$$\begin{aligned} P(D|C=0) &= P(D1|C=0) \times P(D2|C=0) \times P(D3|C=0) \times P(D4|C=0) \\ &= 0.3 \times 0.6 \times 0.6 \times 0.7 \\ &= 0.0756 \end{aligned}$$



The second and third values are 0.6, because the value of that feature in the sample was 0. The listed probabilities are for values of 1 for each feature. Therefore, the probability of a 0 is its inverse: $P(0) = 1 - P(1)$.

Now, we can compute the probability of the data point belonging to this class. Let's take a look at the calculation:

$$P(C=0|D) = P(C=0) P(D|C=0) = 0.75 \times 0.0756 = 0.0567$$

Now, we compute the same values for the class 1:

$$\begin{aligned} P(D|C=1) &= P(D1|C=1) \times P(D2|C=1) \times P(D3|C=1) \times P(D4|C=1) \\ &= 0.7 \times 0.7 \times 0.6 \times 0.9 \\ &= 0.2646 \\ P(C=1|D) &= P(C=1)P(D|C=1) \\ &= 0.25 \times 0.2646 \\ &= 0.06615 \end{aligned}$$



Normally, $P(C=0|D) + P(C=1|D)$ should equal to 1. After all, those are the only two possible options! However, the probabilities are not 1 due to the fact we haven't included the computation of $P(D)$ in our equations here.

As the value for $P(C=1|D)$ is more than $P(C=0|D)$, the data point should be classified as belonging to the class 1. You may have guessed this while going through the equations anyway; however, you may have been a bit surprised that the final decision was so close. After all, the probabilities in computing $P(D|C)$ were much, much higher for the class 1. This is because we introduced a prior belief that most samples generally belong to the class 0.

If the classes had been equal sizes, the resulting probabilities would be much different. Try it yourself by changing both $P(C=0)$ and $P(C=1)$ to 0.5 for equal class sizes and computing the result again.

Applying of Naive Bayes

We will now create a pipeline that takes a tweet and determines whether it is relevant or not, based only on the content of that tweet.

To perform the word extraction, we will be using the spaCy, a library that contains a large number of tools for performing analysis on natural language. We will use spaCy in future chapters as well.



*To get spaCy on your computer, use pip to install the package: `pip install spacy`
If that doesn't work, see the spaCy installation instructions at <https://spacy.io/> for information specific to your platform.*

We are going to create a pipeline to extract the word features and classify the tweets using Naive Bayes. Our pipeline has the following steps:

- Transform the original text documents into a dictionary of counts using spaCy's word tokenization.
- Transform those dictionaries into a vector matrix using the `DictVectorizer` transformer in scikit-learn. This is necessary to enable the Naive Bayes classifier to read the feature values extracted in the first step.
- Train the Naive Bayes classifier, as we have seen in previous chapters.

We will need to create another Notebook (last one for the chapter!) called `ch6_classify_twitter` for performing the classification.

Extracting word counts

We are going to use spaCy to extract our word counts. We still want to use it in a pipeline, but spaCy doesn't conform to our transformer interface. We will need to create a basic transformer to do this to obtain both fit and transform methods, enabling us to use this in a pipeline.

First, set up the transformer class. We don't need to *fit* anything in this class, as this transformer simply extracts the words in the document. Therefore, our fit is an empty function, except that it returns self which is necessary for transformer objects to conform to the scikit-learn API.

Our transform is a little more complicated. We want to extract each word from each document and record True if it was discovered. We are only using the binary features here—True if in the document, False otherwise. If we wanted to use the frequency we would set up counting dictionaries, as we have done in several of the past chapters.

Let's take a look at the code:

```
import spacy
from sklearn.base import TransformerMixin

# Create a spaCy parser
nlp = spacy.load('en')

class BagOfWords(TransformerMixin):
    def fit(self, X, y=None):
        return self

    def transform(self, X):
        results = []
        for document in X:
            row = {}
            for word in list(nlp(document, tag=False, parse=False, entity=False)):
                if len(word.text.strip()): # Ignore words that are just whitespace
                    row[word.text] = True
            results.append(row)
        return results
```

The result is a list of dictionaries, where the first dictionary is the list of words in the first tweet, and so on. Each dictionary has a word as key and the value True to indicate this word was discovered. Any word not in the dictionary will be assumed to have not occurred in the tweet. Explicitly stating that a word's occurrence is False will also work, but will take up needless space to store.

Converting dictionaries to a matrix

The next step converts the dictionaries built as per the previous step into a matrix that can be used with a classifier. This step is made quite simple through the DictVectorizer transformer that is provided as part of scikit-learn.

The DictVectorizer class simply takes a list of dictionaries and converts them into a matrix. The features in this matrix are the keys in each of the dictionaries, and the values correspond to the occurrence of those features in each sample. Dictionaries are easy to create in code, but many data algorithm implementations prefer matrices. This makes DictVectorizer a very useful class.

In our dataset, each dictionary has words as keys and only occurs if the word actually occurs in the tweet. Therefore, our matrix will have each word as a feature and a value of True in the cell if the word occurred in the tweet.

To use DictVectorizer, simply import it using the following command:

```
| from sklearn.feature_extraction import DictVectorizer
```

Putting it all together

Finally, we need to set up a classifier and we are using Naive Bayes for this chapter. As our dataset contains only binary features, we use the `BernoulliNB` classifier that is designed for binary features. As a classifier, it is very easy to use. As with `DictVectorizer`, we simply import it and add it to our pipeline:

```
from sklearn.naive_bayes import BernoulliNB
```

Now comes the moment to put all of these pieces together. In our Jupyter Notebook, set the filenames and load the dataset and classes as we have done before. Set the filenames for both the tweets themselves (not the IDs!) and the labels that we assigned to them. The code is as follows:

```
import os
input_filename = os.path.join(os.path.expanduser("~"), "Data", "twitter", "python_tweets.json")
labels_filename = os.path.join(os.path.expanduser("~"), "Data", "twitter", "python_classes.json")
```

Load the tweets themselves. We are only interested in the content of the tweets, so we extract the text value and store only that. The code is as follows:

```
import json

tweets = []
with open(input_filename) as inf:
    for line in inf:
        if len(line.strip()) == 0: continue
        tweets.append(json.loads(line)['text'])

with open(labels_filename) as inf:
    labels = json.load(inf)

# Ensure only classified tweets are loaded
tweets = tweets[:len(labels)]
```

Now, create a pipeline putting together the components from before. Our pipeline has three parts:

1. The `NLTKBOW` transformer we created.
2. A `DictVectorizer` transformer.
3. A `BernoulliNB` classifier.

The code is as follows:

```
from sklearn.pipeline import Pipeline

pipeline = Pipeline([('bag-of-words', BagOfWords()), ('vectorizer', DictVectorizer()), ('naive-l
```

We can nearly run our pipeline now, which we will do with `cross_val_score` as we have done many times before. Before we perform the data mining, we will introduce a better evaluation metric than the accuracy metric we used before. As we will see, the use of accuracy is not adequate for datasets when the number of samples in each class is different.

Evaluation using the F1-score



When choosing an evaluation metric, it is always important to consider cases where that evaluation metric is not useful. Accuracy is a good evaluation metric in many cases, as it is easy to understand and simple to compute. However, it can be easily faked. In other words, in many cases, you can create algorithms that have a high accuracy but have a poor utility.

While our dataset of tweets (typically, your results may vary) contains about 50 percent programming-related and 50 percent nonprogramming, many datasets aren't as *balanced* as this.

As an example, an e-mail spam filter may expect to see more than 80 percent of incoming e-mails be spam. A spam filter that simply labels everything as spam is quite useless; however, it will obtain an accuracy of 80 percent!

To get around this problem, we can use other evaluation metrics. One of the most commonly employed is called an **f1-score** (also called f-score, f-measure, or one of many other variations on this term).



The F1-score is defined on a per-class basis and is based on two concepts: the precision and recall. The precision is the percentage of all the samples that were predicted as belonging to a specific class, that were actually from that class. The recall is the percentage of samples in the dataset that are in a class and actually labeled as belonging to that class.

In the case of our application, we could compute the value for both classes (python-programming and not python-programming).

Our precision computation becomes the question: *of all the tweets that were predicted as being relevant, what percentage were actually relevant?*

Likewise, the recall becomes the question: *of all the relevant tweets in the data set, how many were predicted as being relevant?*

After you compute both the precision and recall, the f1-score is the harmonic mean of the precision and recall:

To use the f1-score in scikit-learn methods, simply set the scoring parameter to f1. By default, this will return the f1-score of the class with label 1. Running the code on our dataset, we simply use the following line of code:

```
from sklearn.cross_validation import cross_val_score
scores = cross_val_score(pipeline, tweets, labels, scoring='f1')
# We then print out the average of the scores:
import numpy as np
print("Score: {:.3f}".format(np.mean(scores)))
```

The result is 0.684, which means we can accurately determine if a tweet using Python relates to the programming language nearly 70 percent of the time. This is using a dataset with only 300 tweets in it.

Go back and collect more data and you will find that the results increase! Keep in mind that your dataset may differ, and therefore your results would too.



More data usually means a better accuracy, but it is not guaranteed!

Getting useful features from models

One question you may ask is, what are the best features for determining if a tweet is relevant or not? We can extract this information from our Naive Bayes model and find out which features are the best individually, according to Naive Bayes.

First, we fit a new model. While the `cross_val_score` gives us a score across different folds of cross-validated testing data, it doesn't easily give us the trained models themselves. To do this, we simply fit our pipeline with the tweets, creating a new model. The code is as follows:

```
model = pipeline.fit(tweets, labels)
```



Note that we aren't really evaluating the model here, so we don't need to be as careful with the training/testing split. However, before you put these features into practice, you should evaluate on a separate test split. We skip over that here for the sake of clarity.

A pipeline gives you access to the individual steps through the `named_steps` attribute and the name of the step (we defined these names ourselves when we created the pipeline object itself). For instance, we can get the Naive Bayes model:

```
nb = model.named_steps['naive-bayes']  
feature_probabilities = nb.feature_log_prob_
```

From this model, we can extract the probabilities for each word. These are stored as log probabilities, which is simply $\log(P(A|f))$, where f is a given feature.

The reason these are stored as log probabilities is because the actual values are very low. For instance, the first value is -3.486, which correlates to a probability under 0.03 percent. Logarithm probabilities are used in computation involving small probabilities like this as they stop underflow errors where very small values are just rounded to zeros. Given that all of the probabilities are multiplied together, a single value of 0 will result in the whole answer always being 0! Regardless, the relationship between values is still the same; the higher the value, the more useful that feature is.

We can get the most useful features by sorting the array of logarithm probabilities. We want descending order, so we simply negate the values first. The code is as follows:

```
top_features = np.argsort(-nb.feature_log_prob_[1])[:50]
```

The preceding code will just give us the indices and not the actual feature values. This isn't very useful, so we will map the feature's indices to the actual values. The key is the `DictVectorizer` step of the pipeline, which created the matrices for us. Luckily this also records the mapping, allowing us to find the feature names that correlate to different columns. We can extract the features from that part of the pipeline:

```
| dv = model.named_steps['vectorizer']
```

From here, we can print out the names of the top features by looking them up in the `feature_names_` attribute of `DictVectorizer`. Enter the following lines into a new cell and run it to print out a list of the top features:

```
| for i, feature_index in enumerate(top_features):  
|     print(i, dv.feature_names_[feature_index], np.exp(feature_probabilities[1][feature_index]))
```

The first few features include : RT, and even Python. These are likely to be noise (although the use of a colon is not very common outside programming), based on the data we collected. Collecting more data is critical to smoothing out these issues. Looking through the list though, we get a number of more obvious programming features:

```
| 9 for 0.175  
| 14 ) 0.10625  
| 15 ( 0.10625  
| 22 jobs 0.0625  
| 29 Developer 0.05
```

There are some others too that refer to Python in a work context, and therefore might be referring to the programming language (although freelance snake handlers may also use similar terms, they are less common on Twitter).

That last one is usually in the format: *We're looking for a candidate for this job.*

Looking through these features gives us quite a few benefits. We could train people to recognize these tweets, look for commonalities (which give insight into a topic), or even get rid of features that make no sense. For example, the word RT appears quite high in this list; however, this is a common Twitter phrase for retweet (that is, forwarding on someone else's tweet). An expert could decide to remove this word from the list, making the classifier less prone to the noise we introduced by having a small dataset.

Summary

In this chapter, we looked at text mining—how to extract features from text, how to use those features, and ways of extending those features. In doing this, we looked at putting a tweet in context—was this tweet mentioning python referring to the programming language? We downloaded data from a web-based API, getting tweets from the popular microblogging website Twitter. This gave us a dataset that we labeled using a form we built directly in the Jupyter Notebook.

We also looked at reproducibility of experiments. While Twitter doesn't allow you to send copies of your data to others, it allows you to send the tweet's IDs. Using this, we created code that saved the IDs and recreated most of the original dataset. Not all tweets were returned; some had been deleted in the time since the ID list was created and the dataset was reproduced.

We used a Naive Bayes classifier to perform our text classification. This is built upon the Bayes' theorem that uses data to update the model, unlike the frequentist method that often starts with the model first. This allows the model to incorporate and update new data, and incorporate a prior belief. In addition, the naive part allows to easily compute the frequencies without dealing with complex correlations between features.

The features we extracted were word occurrences—did this word occur in this tweet? This model is called bag-of-words. While this discards information about where a word was used, it still achieves a high accuracy on many datasets. This entire pipeline of using the bag-of-words model with Naive Bayes is quite robust. You will find that it can achieve quite good scores on most text-based tasks. It is a great baseline for you, before trying more advanced models. As another advantage, the Naive Bayes classifier doesn't have any parameters that need to be set (although there are some if you wish to do some tinkering).

To extend the work we did in this chapter, first start by collecting more data. You'll need to manually classify these as well, but you'll find some similarities between tweets that might make it easier. For example, there is a field of study called Locality Sensitive Hashes, that determines whether two tweets are similar. Two similar tweets are likely about the same topic. Another method for extending the research is to consider how you would build a model that incorporates the twitter user's history into the equation - in other words, if the user often tweets about python-as-a-programming-language, then they are more likely to be using python in a future tweet.

In the next chapter, we will look at extracting features from another type of data, graphs, in order to make recommendations on who to follow on social media.